Lab3 - Group 45

Lara Faria (106059), Tiago Santos (106329)

Questão 1.1

Função `discrete_state`

A função `discretize_state` mapeia os dados contínuos do simulador para um conjunto finito de estados, permitindo que o agente utilize uma *Q-table*. Para representar as posições verticais do peixe e da barra, decidimos calcular a distância relativa entre os dois em vez de usar as suas coordenadas absolutas. Assim, conseguimos reduzir significativamente a complexidade do problema, focando-se no desvio entre os dois elementos, que é a informação mais importante para a captura.
Escolhemos um intervalo (bin) de 20 pixeis para a posição vertical. Este valor é um bom *trade-off* entre não criar estados em excesso mas ainda conseguir representar certos aspetos importantes da física do jogo, como por exemplo, o tamanho do peixe (20 pixeis) ou distinguir se o peixe está centrado ou a escapar da barra (visto que a altura da barra é de 80 pixeis, o que nesta representação equivale a 4 bins).
Quanto à velocidade, escolhemos um bin de tamanho 0.5 já que as constantes de gravidade e *thrust* são de pequena magnitude (0.15 e 0.35, respetivamente). Assim o agente consegue detetar variações de inércia muito mais cedo, melhorando a sua capacidade de antecipar movimentos face à dinâmica do jogo.
A grande vantagem desta abordagem é a redução drástica do espaço de estados. Se usássemos as posições absolutas do peixe e da barra em simultâneo, combinadas com os *bins* de velocidade, teríamos um universo de 35.625 estados ($25 \times 25 \times 57$). Ao transitar para a distância relativa, comprimimos o problema para cerca de 2.800 estados ($\sim 50 \times 57$). Esta redução de mais de 90% acelera a convergência, garantindo que o agente visita cada par estado-ação vezes suficientes durante o treino.
Em contrapartida, esta simplificação tem duas desvantagens claras. A primeira é a perda de precisão que dificulta microajustes na barra. A segunda, e mais grave, é o desconhecimento do agente em relação aos limites do ecrã. Como não conhece a sua posição absoluta, o agente não sabe quando está encostado ao teto ou ao fundo. Visto que a física do jogo anula a velocidade contra as paredes, tentar subir no topo do ecrã não faz nada, enquanto no meio do ecrã causa um impulso normal. Isto significa que a mesma ação produz resultados diferentes dependendo da posição real da barra, o que inevitavelmente confunde a aprendizagem nas zonas limite.
Apesar destas limitações, prevê-se que o seu impacto seja reduzido, uma vez que a maioria da interação deverá ocorrer no centro do ecrã. Assim, o ganho drástico na velocidade de convergência compensa estas falhas, tornando a abordagem globalmente mais eficaz.

Função `get_value`

No modo de treino, o agente gera um valor aleatório entre 0 e 1. Se este for inferior a ϵ, realiza uma ação ao acaso para descobrir estratégias potencialmente melhores. Caso contrário, ou quando em modo de teste, o agente segue a política *greedy*, selecionando a ação com o menor valor Q para garantir o melhor desempenho possível com base no que já aprendeu.

Q-Learning vs SARSA

A análise do desempenho dos agentes baseia-se nos dados recolhidos durante 5000 episódios de treino e 500 episódios de teste (50 por tipo de peixe), comparando os algoritmos de aprendizagem com a heurística base (*Predictive Agent*). Os resultados quantitativos estão detalhados na Tabela 1 e a evolução dos custos é visível na Figura 1.

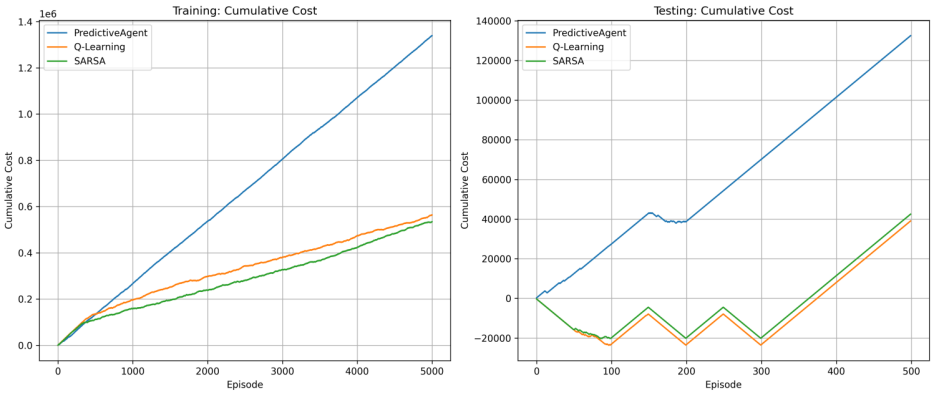

Figura 1: Evolução do custo cumulativo durante as fases de treino (esquerda) e teste (direita)

Agente	**Win Rate (%)**	**Avg. Cost**	**Avg. Steps**
PredictiveAgent	7.6%	265.0	711.6
Q-Learning	**37.6%**	**78.1**	**655.3**
SARSA	36.4%	85.0	696.0

Tabela 1: Resumo comparativo do desempenho médio dos agentes em teste

No gráfico de treino, observa-se que o *Predictive Agent* mantém um crescimento de custo linear e elevado, uma vez que não possui capacidade de adaptação e os seus erros acumulam-se de forma constante.
Em contraste, as curvas do Q-Learning e do SARSA, ainda que comecem com uma inclinação acentuada (fase inicial de exploração), apresentam uma curvatura descendente, sinalizando a convergência das políticas à medida que o agente aprende a minimizar os custos. No que diz respeito à taxa de convergência, ambos os algoritmos começam a estabilizar a sua aprendizagem de forma evidente por volta do episódio 3000.
É igualmente visível que o algoritmo SARSA apresenta um custo acumulativo de treino ligeiramente inferior ao Q-Learning. Sendo um algoritmo *on-policy*, o SARSA aprende o valor da política que está efetivamente a seguir, incluindo os erros da exploração ϵ-greedy. Isto torna-o mais cauteloso, evitando posições de risco onde uma ação aleatória pudesse resultar num custo elevado durante o treino.

Na fase de teste (onde $\epsilon = 0$ e a execução é estritamente *greedy*), a superioridade dos métodos de aprendizagem é evidente. O gráfico de teste apresenta várias oscilações porque a dificuldade do jogo muda drasticamente conforme o tipo de peixe que o agente está a tentar apanhar em cada momento. O Q-Learning destaca-se como o melhor agente, obtendo uma *Win Rate* de 37.6% e a maior eficiência temporal (655.3 passos). Por ser um algoritmo *off-policy*, foca-se em aprender a estratégia ótima, ignorando a componente aleatória da sua própria exploração. Esta abordagem mais direta permite-lhe ser mais preciso e eficiente durante o teste. Em contrapartida, o algoritmo SARSA obteve um resultado ligeiramente inferior (36,4%) por ser mais cuidadoso. Como este aprende com base no que realmente faz durante o treino (incluindo as ações aleatórias), acaba por desenvolver uma política mais segura e conservadora. Embora esta cautela ajude a evitar erros graves no treino, torna-o menos eficaz que a precisão absoluta do Q-Learning quando a exploração é desligada no teste.

Questão 1.2

1

A robustez do agente Q-Learning foi avaliada através de 10 execuções independentes, utilizando diferentes *random seeds*. Os dados consolidados de desempenho médio estão resumidos na Tabela 2.

Métrica	**Média (μ)**
Win Rate (%)	37.1%
Avg. Cost	81.7
Avg. Steps	695.0

Tabela 2: Métricas Médias do Q-Learning (10 Runs)

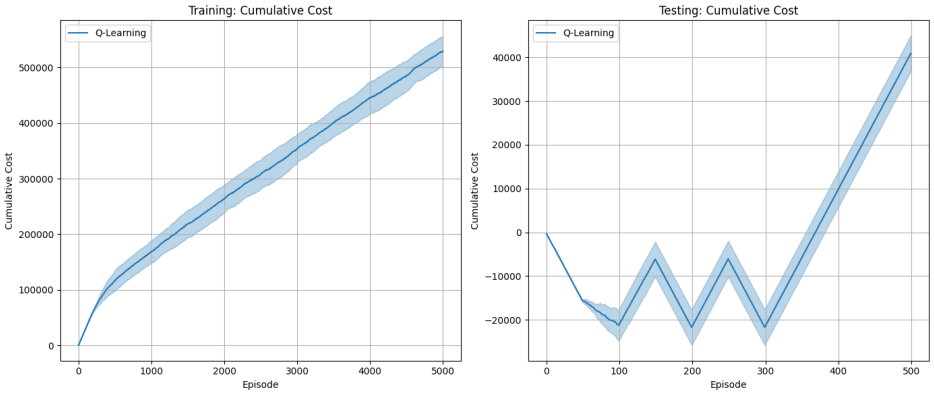

Figura 2: Custo cumulativo de Treino (esquerda) e Teste (direita) para o agente Q-Learning.

A robustez do treino é confirmada pela elevada proximidade dos resultados em relação à média. As *win rates* situam-se consistentemente entre os 31.5% e os 34.4%, resultando num desvio padrão muito reduzido. No gráfico de treino (Figura 2, esquerda), a área sombreada azul, que representa a variação estatística entre as *seeds*, é estreita. Esta consistência prova que o algoritmo é fiável e que a política converge para uma solução estável, independentemente das condições iniciais de cada treino.

- No gráfico de treino (Figura 2, esquerda), observa-se que o desvio padrão é quase nulo no início (uma vez que todas as execuções partem de um custo zero) e vai alargando progressivamente, por ser uma métrica cumulativa. O verdadeiro indicador de robustez é o facto de este alargamento ser contido, formando uma margem estreita e consistente que acompanha a trajetória média, sem divergências em nenhuma das 10 corridas. Além disso, a inclinação da curva diminui gradualmente ao longo dos episódios, provando que o agente abandona a fase inicial de aprendizagem acelerada, onde o custo acumulado cresce depressa devido à exploração inicial (ϵ elevado), e aprende a otimizar a estratégia e minimizar as penalizações.
- No gráfico de teste (Figura 2, direita), as oscilações visíveis devem-se à avaliação sequencial de diferentes tipos de peixe (50 episódios para cada tipo). As descidas na curva representam períodos em que o agente domina a dinâmica do peixe atual, acumulando recompensas (custos negativos), enquanto as subidas refletem peixes com movimentos mais complexos ou erráticos, onde a política encontra maiores dificuldades. O facto de a banda do desvio padrão se manter extremamente colada à média, mesmo durante estas transições bruscas de dificuldade, valida definitivamente o Q-Learning como uma solução robusta e imune a condições de inicialização aleatórias.

2

A política final do agente Q-Learning não é ótima por três razões:

1. Win Rate: O indicador mais evidente é a *Win Rate* de 37.1%. Num cenário de política ótima, o agente deveria ser capaz de capturar o peixe na grande maioria dos episódios. O facto de falhar em mais de 60% das vezes demonstra que a estratégia aprendida ainda não é eficaz contra peixes com dinâmicas mais complexas.

2. Erro de Discretização: O ambiente do jogo é contínuo, mas o agente funciona num espaço de estados discretizado. Esta simplificação cria um "erro de quantização", ou seja, o agente não consegue distinguir pequenas variações de movimento que exigiriam pequenos ajustes. Portanto, o agente pode ter encontrado a melhor política possível para o modelo discreto, mas esta será sempre uma aproximação inferior à política ótima do modelo contínuo.

3. Exploração e Convergência: Para garantir a otimalidade (Q^*), o Q-Learning exige que todos os estados e ações sejam explorados exaustivamente. Com 5000 episódios e um decaimento de ϵ relativamente rápido, é provável que o agente tenha convergido para um local ótimo. Ou seja, ele aprendeu uma estratégia que funciona para os casos mais comuns, mas não explorou o suficiente para encontrar soluções ideais para situações raras ou extremas.

Concluindo, embora a política seja robusta (consistente) e eficiente face à *baseline*, os limites impostos pela discretização e a taxa de sucesso observada confirmam que o agente ainda está distante de uma política perfeitamente ótima.

3

Para perceber o impacto dos parâmetros de exploração no algoritmo Q-Learning, fizemos duas experiências, alterando um parâmetro de cada vez face à configuração original (*Baseline*: $\epsilon = 0.1$, $decay = 0.995$). Os resultados médios de 10 execuções estão na Tabela 3.

Configuração	ϵ_0	**Decay**	**Test Win Rate**	**Avg Cost**	**Avg Steps**
Baseline	0.1	0.995	37.1%	81.7	695.0
Exp. A	1.0	0.995	36.5%	85.1	704.0
Exp. B	0.1	0.999	36.4%	85.2	704.8

Tabela 3: Comparação de métricas variando os parâmetros de exploração

1. **Experiência A: Aumento do ϵ inicial para 1.0**

   Com esta experiência pretendíamos testar o que acontecia se o agente começasse o treino a explorar de forma 100% aleatória. No início, os resultados foram muito fracos (quase não houve vitórias nos primeiros 300 episódios) porque o agente tomou muitas ações aleatórias. Apesar de ter recuperado no fim do treino ($\sim$33.5% de *Win Rate*), o desempenho no teste foi pior do que a *baseline* (o *Avg Cost* subiu para 85.1 e os passos para 704.0). Isto aconteceu porque a taxa de decaimento se manteve rápida (0.995): o agente teve um início caótico e parou de explorar demasiado cedo, ficando preso em caminhos que não eram os ideais.

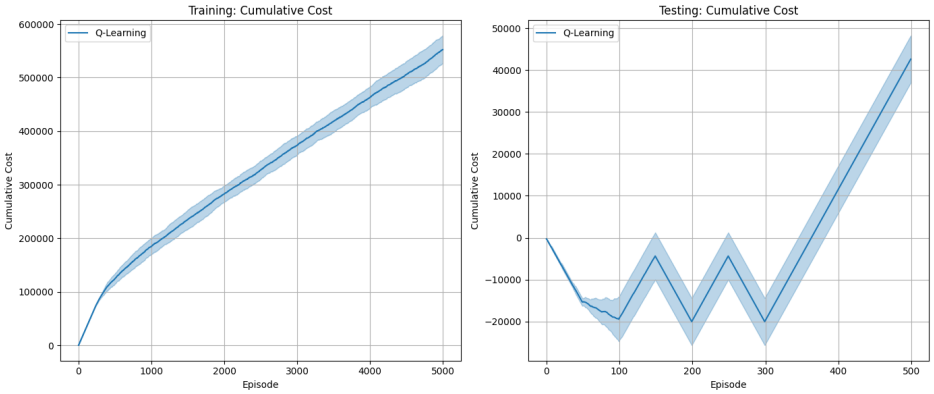

Figura 3: Custo cumulativo para o agente Q-Learning, com epsilon=1.0

2. **Experiência B: Decay mais lento de 0.999**

   O objetivo nesta experiência foi isolar o efeito da taxa de decaimento e obrigar o agente a explorar o ambiente durante mais tempo. Como era de esperar, a aprendizagem foi mais lenta, porque ele continuou a tomar decisões aleatórias quase até ao final dos 5000 episódios. Como mantivemos o ϵ inicial baixo (10%), o agente passou muito tempo a explorar de forma contida e pouco tempo a aproveitar o que já sabia (*exploitation*). Assim, a Q-Table não teve tempo suficiente para otimizar totalmente a política e o resultado foi o pior desempenho na fase de teste, com um *Avg Cost* de 85.2 e 704.8 passos médios.

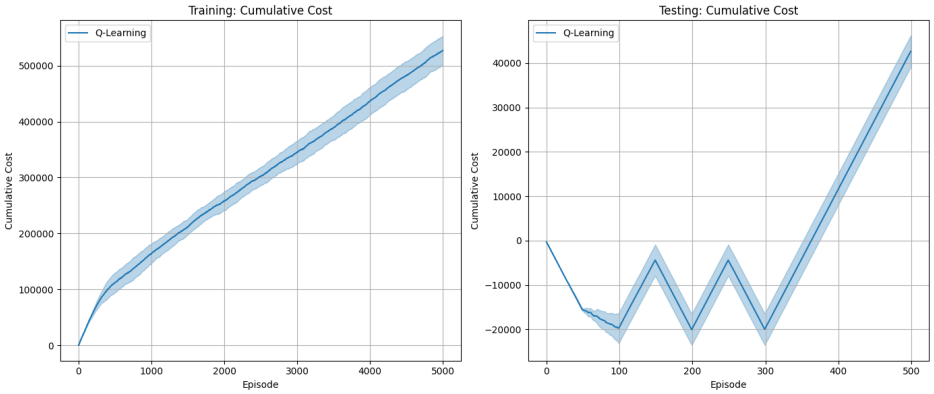

Figura 4: Custo cumulativo para o agente Q-Learning, com epsilon_decay = 0.999

Conclusão: Estes resultados mostram que, para o nosso espaço de estados e com um limite de 5000 episódios, a *Baseline* original é a que funciona melhor (ainda que possam existir combinações destes valores com melhor desempenho). Começar com apenas 10% de exploração é suficiente para descobrir o essencial do ambiente, e um *decay* rápido (0.995) permite que o agente passe a maior parte do seu tempo focado em melhorar e afinar a estratégia final para o teste.

4

Para esta questão, o agente foi treinado durante 5000 episódios utilizando exclusivamente o peixe *Sturgeon* enquanto a fase de teste incluiu todos os tipos de peixes disponíveis para avaliar a capacidade de adaptação do modelo. Os resultados detalhados desta comparação podem ser observados na Tabela 4.

A experiência demonstra claramente o compromisso entre a especialização e a capacidade de generalização. Treinar o agente num único tipo de peixe resulta num fenómeno clássico de *overfitting*. Embora

Métrica	Configuração Original	Sturgeon Only
Win Rate	37.6%	30.4%
Avg. Cost	78.1	121.4
Avg. Steps	655.3	705.4

Tabela 4: Comparação de métricas entre o agente geral e o especialista

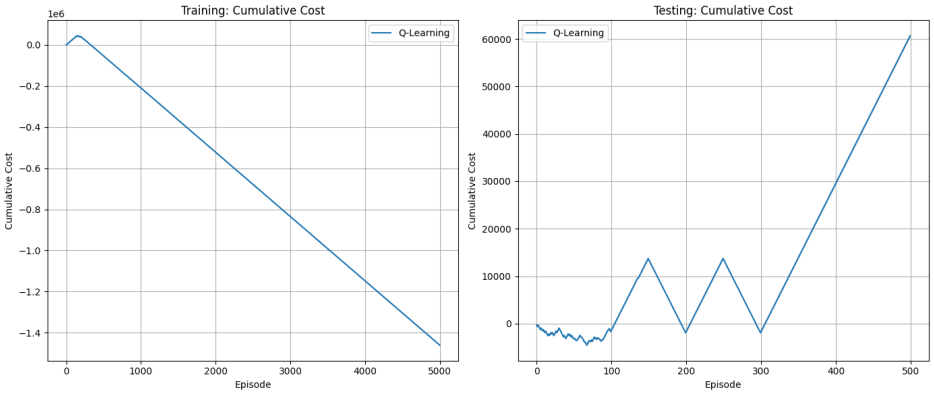

Figura 5: Custo cumulativo de treino e teste para o agente treinado apenas com Sturgeon

o agente tenha sucesso em casos semelhantes, o seu rendimento global cai drasticamente quando as dinâmicas do ambiente mudam na fase de teste.

Relativamente aos custos acumulados durante o treino, o modelo que utilizou apenas o Sturgeon apresenta valores muito inferiores aos do modelo original. Esta diferença justifica-se pelo facto de o cenário de aprendizagem ser constante e previsível sem as variações introduzidas pelos outros peixes. Isto permitiu que o agente atingisse uma taxa de vitória de 96.7% no final do treino. O agente acabou por memorizar o padrão de movimento deste peixe específico em vez de aprender regras gerais. Na configuração original o custo era mais elevado porque a diversidade de movimentos obrigava o agente a um esforço de adaptação constante para conseguir estabilizar a sua estratégia mais generalista.

No entanto a situação inverte-se nos resultados de teste onde usamos todos os peixes. O custo médio do especialista subiu para 121.4 comparado com os 78.1 do modelo original o que prova que o agente sofreu uma especialização excessiva. Quando o peixe no teste não é o Sturgeon o agente tenta aplicar a mesma tática e acaba por falhar muito mais vezes. No gráfico de teste da Figura 5 é visível o custo a subir rapidamente durante a avaliação final.

Em relação à média de passos, o modelo especialista também foi menos eficiente atingindo 705.4 passos contra os 655.3 do modelo original. Isto indica que, ao deparar-se com dinâmicas de movimento para as quais não foi treinado, o agente entra frequentemente em estados que ainda não explorou ou aplica uma política desajustada. Consequentemente, a barra afasta-se do alvo, prolongando a duração dos episódios.

Em conclusão forçar o treino num único tipo de peixe cria um agente que é muito bom num cenário controlado mas que perde a utilidade noutros ambientes. A configuração original demonstra ser muito mais robusta porque obriga o agente a aprender regras de movimento que funcionam de forma mais geral para todos os peixes.

Questão 1.3

A ideia implementada para melhorar o agente consistiu em aumentar a precisão da representação do estado através da redução do tamanho do bin de 20 para 10. Esta alteração permite ao agente perceber mudanças mais subtis na posição e velocidade do peixe. Os resultados desta alteração estão detalhados na Tabela 5.

Agente	**Win Rate (%)**	**Avg. Cost**	**Avg. Steps**
PredictiveAgent	6.8%	269.1	731.0
Q-Learning	36.0%	86.4	693.9
SARSA	37.6%	76.1	674.5

Tabela 5: Resultados obtidos com a redução do bin para 10

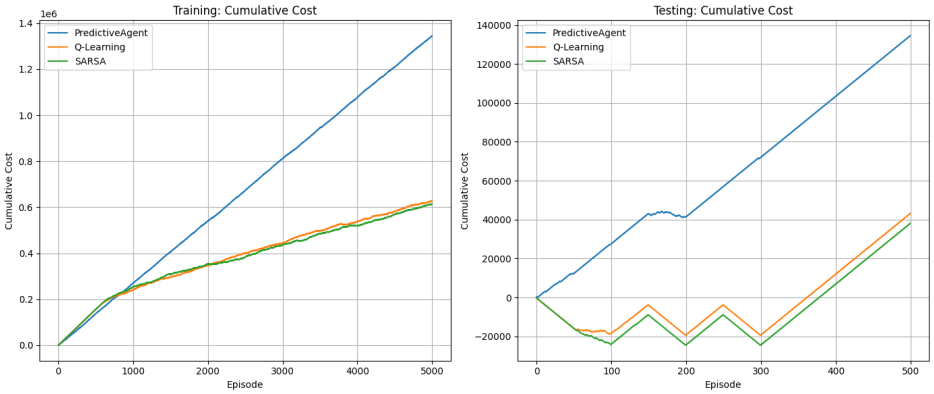

Figura 6: Evolução do custo cumulativo com a nova configuração de bins

A análise dos resultados revela um impacto interessante na performance dos agentes. Ao contrário da configuração original, o SARSA passou a ser o melhor algoritmo, atingindo uma taxa de sucesso de 37.6% e o custo médio mais baixo. O Q-Learning apresentou uma ligeira descida na sua eficácia em comparação com o bin de 20.
Esta diferença acontece porque ao diminuirmos o tamanho do bin estamos a aumentar o tamanho da Q-table. Com mais estados para explorar o agente precisa de mais tempo de treino para preencher todas as células de forma correta. O SARSA parece ter lidado melhor com este aumento de resolução no tempo disponível de 5000 episódios. No gráfico de custo cumulativo da Figura 6 podemos ver que a convergência é mais lenta no início devido ao maior espaço de estados a explorar.
Em conclusão a alteração do bin para 10 trouxe maior precisão ao SARSA mas mostrou que o aumento da complexidade do estado exige um maior equilíbrio com o tempo de treino disponível para garantir a otimalidade de todos os algoritmos.

Para além da redução do tamanho do bin identificamos outras oportunidades de melhoria que poderiam aumentar a eficácia e a inteligência do agente, no entanto, não foram implementadas. Estas sugestões focam-se principalmente na forma como o agente percebe o ambiente e como é incentivado a aprender.

- Uma das limitações da abordagem de distância relativa descrita na questão 1.1, é o facto de o agente perder a noção da sua posição absoluta no ecrã. Embora saber a distância ao peixe seja fundamental o agente não sabe quão perto está do topo ou do fundo do simulador. Isto é problemático porque as ações disponíveis são limitadas pelas bordas do ecrã. Se o agente estiver no limite superior e o peixe subir ele não pode realizar mais movimentos nessa direção. Uma forma de resolver isto sem aumentar a complexidade na Q-table seria adicionar indicadores binários ao estado. Estes indicadores avisariam o agente quando a barra estivesse a menos de uma certa distância dos limites. Assim o agente poderia aprender a posicionar-se de forma mais preventiva evitando ficar encurralado nas extremidades do jogo.

- Atualmente o sistema de recompensas é muito simples atribuindo valores fixos de vitória ou derrota. Isto cria o que se chama de recompensas esparsas onde o agente demora muito tempo a perceber se uma ação foi boa ou má. Podíamos implementar uma função de recompensa proporcional à distância. Em cada passo o agente receberia uma pequena pontuação positiva se estivesse a diminuir a distância para o peixe e uma penalização se estivesse a afastar-se. Isto guiaria o agente de forma muito mais suave durante o treino permitindo uma convergência mais rápida para a política ideal.